

Cluster: IPCV



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

VisionProctor

Submitted by

Alex Shrestha (080BCT012)

Darpan Giri (080BCT024)

Mahesh Bhandari (080BCT045)

A

BE MINOR PROJECT REPORT

**SUBMITTED TO THE DEPARTMENT OF ELECTRONICS & COMPUTER
ENGINEERING IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF BACHELOR OF ENGINEERING IN
COMPUTER ENGINEERING**

**DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING
LALITPUR, NEPAL**

26 AUGUST, 2026

Declaration

We hereby declare that the work presented in this report entitled **VisionProctor** is our own work and has been carried out under the supervision of Mr. Prakash Chandra Prasad, Assistant Professor, Department of Electronics and Computer Engineering, Pulchowk Campus, and has not been submitted elsewhere for the award of any degree, diploma, or other similar title. We declare that this work is our own. If any part of the work is not our own, we have indicated it by acknowledging the source of that part or those part of the work with proper citations.

Alex Shrestha (080BCT012)

Darpan Giri (080BCT024)

Mahesh Bhandari (080BCT045)

Date: 26 AUGUST, 2026

Approval

This report entitled **VisionProctor**, submitted by Alex Shrestha (080BCT012), Darpan Giri (080BCT024), and Mahesh Bhandari (080BCT045), in partial fulfilment of the requirements for the minor project of the Bachelor's Degree in Electronics and Computer Engineering, has been examined and is hereby approved.

Mr. Prakash Chandra Prasad
Supervisor
Department of Electronics and Computer Engineering
Pulchowk Campus

Copyright

The authors have agreed that the library, Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering may make this project report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this report for scholarly purpose may be granted by the professor who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project was done. It is understood that the recognition will be given to the author of this report and to the Department of Electronics & Computer Engineering, Pulchowk Campus, and Institute of Engineering in any use of the material of this project. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering and authors written permission is prohibited.

Abstract

The rapid shift towards online learning and remote examinations has exposed the limitations of traditional human invigilation, which is costly, difficult to scale, and prone to inconsistency. In this project, **VisionProctor**, an AI-based online proctoring system is proposed to automate the detection of common exam malpractices in real time. A pretrained MediaPipe Face Mesh model (468 facial landmarks) is used to perform gaze estimation and head-pose tracking, and a YOLOv8 object detection model, fine-tuned on a custom dataset, is used to detect mobile phones and multiple persons within the webcam frame. In addition, a browser-level monitoring module is implemented to detect tab-switching and other suspicious window activity during the exam session. Each module independently monitors its respective signal, and suspicious events are logged in real time for administrator review. The system is designed as a lightweight, CPU-friendly, and modular alternative to expensive commercial proctoring solutions such as Proctorio and Respondus, with the aim of reducing false positives through temporal smoothing of individual detection signals.

Keywords: Online Proctoring, Computer Vision, MediaPipe, YOLOv8, Object Detection, Gaze Tracking, Head Pose Estimation, Exam Integrity.

Acknowledgements

The Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, is gratefully acknowledged for providing the opportunity, resources, and academic environment required to carry out this project. Sincere gratitude is extended to the project supervisor, Mr. Prakash Chandra Prasad, Assistant Professor, Department of Electronics and Computer Engineering, for his continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. His technical insights and encouragement helped refine the ideas presented and improve the overall quality of this work.

The friends and also some faculty members of the department are also thanked for their valuable support and guidance during the course of this project. Their knowledge and suggestions contributed significantly to the learning experience and to the successful completion of this work.

Finally, gratitude is extended to everyone who directly or indirectly supported this work. Their encouragement and assistance have been valuable throughout this journey.

Alex Shrestha (080BCT012)

Darpan Giri (080BCT024)

Mahesh Bhandari (080BCT045)

Table of Contents

Declaration	i
Approval	ii
Copyright	iii
Abstract	iv
Acknowledgements	v
Table of Contents	viii
List of Figures	ix
List of Tables	x
Abbreviations	xi
Chapter 1 .Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.3.1 General Objective	1
1.3.2 Specific Objectives	2
1.4 Rationale	2
1.5 Scope	2
1.6 Limitations	2
1.7 Outline of the Report	3
Chapter 2 .Literature Review	4
2.1 Review of Related Theory	4
2.1.1 Facial Landmark Detection (MediaPipe Face Mesh)	4
2.1.2 Head Pose Estimation	4

2.1.3	You Only Look Once (YOLO) Object Detection	4
2.1.4	Event Logging and Temporal Smoothing	5
2.1.5	Cross-Entropy Loss and Model Training	5
2.2	Review of Related Works	5
2.3	Gap Analysis	6
Chapter 3	.Methodology	7
3.1	Project Approach	7
3.2	System Architecture	7
3.2.1	Client-Server Architecture	7
3.2.2	AI Processing Pipeline	9
3.3	System Design / Modules	9
3.3.1	Gaze and Head Pose Tracking Module	9
3.3.2	Object and Multiple-Person Detection Module	10
3.3.3	Tab-Switching Detection Module	10
3.3.4	Event Logging Engine	10
3.4	Technology Stack	11
3.5	Development Methodology	11
3.6	Data Handling	11
3.7	Testing and Validation	11
3.8	Deployment Setup	11
Chapter 4	.Implementation Details	13
4.1	Requirements Specification	13
4.1.1	Functional Requirements	13
4.1.2	Non-Functional Requirements	13
4.2	System Overview	14
4.3	Development Environment	15
4.4	System Implementation	16
4.4.1	Dataset	16
4.4.2	Dataset Preprocessing	16
4.4.3	Model Selection	16
4.4.4	Training Configuration (YOLOv8 Fine-Tuning)	17
4.4.5	Real-Time Performance Considerations	17
4.5	Interface and Visualization	18
4.5.1	Security Measures	18

4.5.2	Actors and Interfaces	18
4.6	Deployment and Execution	18
4.7	Challenges and Solutions	18
Chapter 5	.Results and Discussion	20
5.1	Results	20
5.1.1	Object Detection Performance	20
5.1.2	Calibration: Gaze and Head Pose Tracking	21
5.1.3	Qualitative Results: Gaze and Head Pose Tracking	23
5.1.4	Qualitative Results: Phone and Multiple-Person Detection	24
5.1.5	Qualitative Results: Tab-Switch Detection	24
5.1.6	Qualitative Results: Person Absent	25
5.1.7	Event Log Summary	26
5.2	Discussion	26
5.3	Comparative Analysis	27
Chapter 6	.Conclusions and Recommendations	28
6.1	Conclusions	28
6.2	Recommendations / Future Enhancements	28
References		30
Appendix 1		31
1.1	Sample Annotated Dataset Images	31

List of Figures

3.1	Agile development methodology	7
3.2	VisionProctor system architecture	8
4.1	Use case diagram for VisionProctor	14
4.2	Activity diagram for VisionProctor	15
5.1	Precision-recall curve for the fine-tuned YOLOv8 model	20
5.2	Training and validation loss curves (box, classification, and DFL losses) and object detection performance metrics (precision, recall, mAP@50, and mAP@50–95) during YOLOv8 fine-tuning	20
5.3	Confusion matrix for the fine-tuned YOLOv8 model on the test set . . .	21
5.4	Eye calibration using the four laptop-screen corner points as gaze refer- ence endpoints	22
5.5	Head-pose calibration using center, left, and right reference head images	22
5.6	Sample frames with head-pose estimation	23
5.7	Sample frames with gaze deviation	23
5.8	Sample detection result: mobile phone	24
5.9	Live tab-switch detection	24
5.10	Tab-switch detection snapshot	24
5.11	Person-absent snapshot	25
5.12	Logged suspicious events over time for a sample exam session	26
1.1	Sample annotated images from the custom object-detection dataset . . .	31

List of Tables

2.1	Comparative summary of existing proctoring approaches	6
4.1	Dataset summary	16
5.1	YOLOv8 fine-tuning results	21

Abbreviations

AI	Artificial Intelligence
CNN	Convolutional Neural Network
YOLO	You Only Look Once
API	Application Programming Interface
JWT	JSON Web Token
REST	Representational State Transfer
COCO	Common Objects in Context
FPS	Frames Per Second
GPU	Graphics Processing Unit
CPU	Central Processing Unit
SDLC	Software Development Life Cycle
MVC	Model-View-Controller
UI	User Interface
PnP	Perspective-n-Point
mAP	Mean Average Precision

Chapter 1. Introduction

1.1. Background

The post-pandemic era has witnessed a rapid and sustained adoption of online learning and remote examinations across academic institutions. While this shift has improved accessibility and flexibility, it has simultaneously introduced significant challenges in maintaining examination integrity. Traditional human invigilation, although effective in physical classrooms, becomes costly and impractical to scale when examinations are conducted remotely for large cohorts of students.

Common integrity risks in remote examinations include the use of unauthorized devices such as mobile phones, consultation of external material, presence of unauthorized persons in the exam environment, and switching between browser tabs or applications to access unauthorized resources. Manual monitoring of hundreds of live video feeds simultaneously is infeasible, which creates a strong need for automated, AI-driven proctoring solutions capable of providing real-time, objective, and scalable monitoring.

1.2. Problem Statement

Existing online examination platforms largely lack integrated, real-time mechanisms for detecting malpractice. Where automated proctoring tools do exist, they are often expensive, closed-source commercial products (e.g., Proctorio, Respondus) that are inaccessible to smaller institutions, or academic prototypes that detect only a narrow set of behaviours (e.g., gaze direction alone, or object detection alone). Furthermore, many existing systems suffer from high false-positive rates under varying lighting conditions and webcam quality, which reduces their practical reliability.

1.3. Objectives

1.3.1. General Objective

The general objective of this project is to design and implement a lightweight, AI-based online proctoring system capable of monitoring multiple behavioural signals in real time to support exam integrity.

1.3.2. Specific Objectives

The specific objectives of this minor project are as follows:

- To track a candidate's eye gaze and head pose in real time using a pretrained facial landmark detection model.
- To detect mobile phones and multiple persons in the exam video feed using a YOLOv8 object detection model fine-tuned on a custom dataset.
- To implement a browser tab-switching detection module that flags window or tab-focus changes during the exam.
- To design an event logging and alerting mechanism that flags suspicious behaviour detected by each module for administrator review.
- To build a scalable client-server system that supports real-time video streaming, exam management, and reporting of proctoring analytics.

1.4. Rationale

Given the infeasibility of manual invigilation at scale and the high cost of existing commercial proctoring tools, an accessible and modular alternative is required. VisionProctor is motivated by the need for a system that (a) combines multiple independent behavioural signals rather than relying on a single cue, (b) runs efficiently on commodity CPU hardware without specialized sensors, and (c) reduces false alarms through temporal smoothing, thereby making automated proctoring more practical for smaller academic institutions.

1.5. Scope

The scope of this project covers the design and implementation of a real-time, browser-based AI proctoring system for single-candidate online examinations. It includes gaze and head-pose tracking, object and person detection, and tab-switch monitoring, each independently logging and flagging suspicious events, along with a web-based front end for students and an administrative interface for exam creators. Future extensions such as face identity verification, audio-based cheating detection and mouth-movement analysis are outlined as potential future work.

1.6. Limitations

1. Gaze and head-pose estimation accuracy can degrade under poor or uneven lighting conditions.

2. Partial occlusion of the face or the monitored object (e.g., a partially hidden phone) can reduce detection reliability.
3. Running both the landmark model and the object detector in real time can be demanding on lower-end CPUs without further optimization.
4. Natural head movements or momentarily looking away from the screen may still trigger false alarms despite temporal smoothing.
5. The current system does not verify candidate identity; it focuses solely on behavioural monitoring during the exam session.

1.7. Outline of the Report

The remainder of this report is organized as follows. Chapter 2 reviews related theory and existing proctoring systems and identifies the gap addressed by this project. Chapter 3 presents the system methodology, architecture, and technology stack. Chapter 4 describes the implementation details, including requirement specification, development environment, and the challenges encountered. Chapter 5 presents the experimental results and discussion. Chapter 6 concludes the report and outlines recommendations for future work.

Chapter 2. Literature Review

2.1. Review of Related Theory

2.1.1. Facial Landmark Detection (MediaPipe Face Mesh)

MediaPipe Face Mesh [1] is a pretrained, lightweight machine learning model that estimates 468 3D facial landmarks from a single RGB image in real time, without requiring specialized hardware such as depth sensors. Alternative facial behaviour analysis toolkits, such as OpenFace 2.0 [10], offer comparable landmark and head-pose estimation capabilities; MediaPipe Face Mesh was adopted in this project for its lightweight, real-time performance on CPU-only hardware. In VisionProctor, this model is used purely for behavioural monitoring, the geometric relationship between eye and iris landmarks is used to estimate gaze direction, while the relative position of landmarks such as the nose tip, eye corners, and chin is used to estimate head pose (yaw, pitch, and roll).

2.1.2. Head Pose Estimation

Head pose is commonly estimated by solving the Perspective-n-Point (PnP) problem, which relates a set of known 3D facial landmark positions to their corresponding 2D projections in the image plane:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R \mid t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (2.1)$$

where K denotes the camera intrinsic matrix, R and t denote the rotation and translation of the head relative to the camera, and (X, Y, Z) denote 3D landmark coordinates. The resulting rotation matrix R is decomposed into yaw, pitch, and roll angles, which are thresholded to flag sustained off-screen attention.

2.1.3. You Only Look Once (YOLO) Object Detection

YOLO [3] formulates object detection as a single regression problem, directly predicting bounding boxes and class probabilities from full images in one forward pass through the network, which makes it well suited to real-time applications. YOLOv8 [2], the version used in this project, is pretrained on the COCO dataset [4] and is capable of detecting common classes such as *person* and *cell phone*; it is further fine-tuned on a custom-labelled dataset to improve detection accuracy specifically for the exam-proctoring use

case.

2.1.4. Event Logging and Temporal Smoothing

Each detection module (gaze/head pose, object/person, tab switch) independently raises an event when its respective condition is met, such as sustained gaze deviation, a detected mobile phone, more than one person in frame, or a tab-switch action. To avoid false alarms caused by single-frame noise or momentary detections, each module applies temporal smoothing before logging an event, for instance by requiring a behaviour to persist for a minimum duration, or by smoothing the raw per-frame signal (e.g., via an exponential moving average) before comparing it against a threshold. Flagged events are timestamped, logged, and made available to the administrator for review.

2.1.5. Cross-Entropy Loss and Model Training

Fine-tuning of the YOLOv8 detection head uses a composite objective combining classification loss, localization (bounding box) loss, and objectness loss. The classification component follows the standard cross-entropy formulation:

$$L_{cls} = - \sum_i y_i \log(p_i) \quad (2.2)$$

where p_i denotes the predicted class probability and y_i denotes the ground-truth label.

2.2. Review of Related Works

Automated proctoring has been explored both by commercial vendors and academic researchers. Commercial platforms such as Proctorio [5] combine face detection, eye tracking, and browser-lockdown features to provide fully automated monitoring, but are often expensive and have been reported to struggle with reliable object detection and head-pose estimation under varied conditions. Respondus [6] similarly combines webcam monitoring and browser lockdown to prevent access to unauthorized applications, but remains closed-source with limited object-detection capability. Academic work, such as the CNN-based proctoring approach of Chen and Alvarez [7] and the multi-cue automated proctoring framework of Atoum et al. [9], has reported good accuracy (approximately 91% for the former) for detecting suspicious activity, but such approaches typically focus on a narrower set of behavioural signals and do not integrate browser-level monitoring such as tab-switch detection.

Table 2.1: Comparative summary of existing proctoring approaches

Author / Approach	Method	Strengths	Limitations
Proctorio	Face detection, eye tracking, browser monitoring	Fully automated	High cost; fails with object detection & head pose tracking
Chen & Alvarez [7]	CNN-based proctoring	Good accuracy (~91%)	Focuses mainly on suspicious activity; no object detection; no tab monitoring
Respondus	Webcam monitoring + face detection + browser lockdown	Prevents opening other applications via LockDown Browser	Closed source; limited object detection

2.3. Gap Analysis

A review of existing systems reveals a set of recurring limitations: most systems detect only one or two behavioural cues (either gaze/pose *or* objects, rarely both), suffer from a high rate of false positives under varying lighting conditions, and are expensive commercial services.

VisionProctor is positioned to address these gaps by combining:

- MediaPipe’s 468-point facial landmark model for real-time, CPU-efficient gaze and head-pose estimation.
- A fine-tuned YOLOv8 model for detecting mobile phones and multiple persons.
- Independent monitoring of gaze/pose, object/person, and browser tab-switch events, each logged and flagged in real time.
- A lightweight, CPU-friendly design suitable for real-time deployment at lower cost than commercial alternatives.

The overall design goal is to reduce false positives in each detection module through temporal smoothing, rather than relying on a single instantaneous reading.

Chapter 3. Methodology

3.1. Project Approach

This project follows the **Agile** software development methodology. Agile allows for iterative development and testing of individual modules (gaze tracking, object detection, tab-switch monitoring, event logging) and supports incorporation of feedback after each testing cycle. This approach was found suitable given the exploratory nature of tuning detection thresholds and fusion weights.

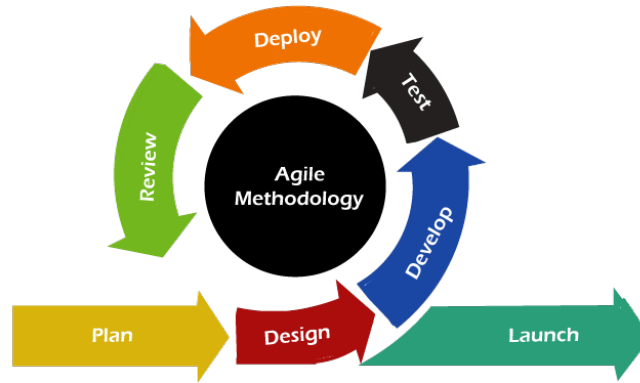


Figure 3.1: Agile development methodology

3.2. System Architecture

3.2.1. Client-Server Architecture

VisionProctor follows a client-server architecture. The client, built with Next.js, captures the candidate's webcam feed and streams it to the backend over an encrypted connection (HTTPS/WebRTC). The FastAPI backend hosts the AI processing pipeline, which performs frame preprocessing, gaze/head-pose estimation, and object and person detection, before returning results and alerts to the client and storing session data in a PostgreSQL database.

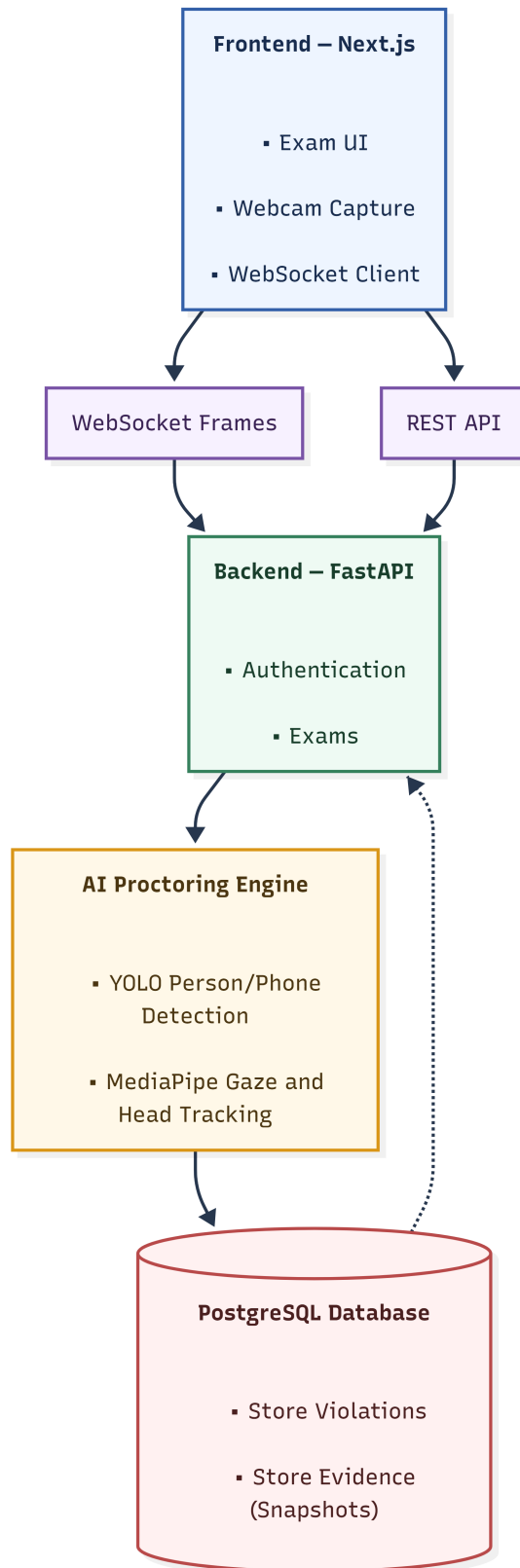


Figure 3.2: VisionProctor system architecture

3.2.2. AI Processing Pipeline

The AI processing pipeline within the backend consists of the following stages:

1. **Image Processing:** Frame capture, resizing, normalization, and denoising using OpenCV.
2. **Gaze / Head Pose Estimation:** MediaPipe Face Mesh landmark extraction, followed by geometric computation of gaze direction and head-pose angles.
3. **Object / Person Detection:** Fine-tuned YOLOv8 inference to detect mobile phones and count persons in frame.
4. **Event Logging Engine:** Reception of flagged events from each detection module, timestamping and logging of events, and triggering of alerts when a module's detection threshold is exceeded.

3.3. System Design / Modules

3.3.1. Gaze and Head Pose Tracking Module

This module uses the pretrained MediaPipe Face Mesh model to extract 468 facial landmarks per frame in real time. Eye landmark geometry is used to estimate gaze direction (on-screen vs. off-screen), while a subset of stable facial landmarks is used to solve the head-pose estimation problem (Equation 2.1), yielding yaw, pitch, and roll angles. No fine-tuning was required for this module, since the pretrained model generalizes well across poses and lighting conditions.

3.3.1.1. Calibration Procedure

Because absolute eye-landmark and head-pose readings vary from candidate to candidate (due to differences in seating position, webcam placement, face geometry, and screen size), a short one-time calibration step is performed at the start of each exam session before gaze and head-pose thresholds are applied.

For **eye/gaze calibration**, the candidate is asked to look, in turn, at the four corner points (endpoints) of the laptop screen – top-left, top-right, bottom-left, and bottom-right. At each of these four fixation points, the eye and iris landmark positions returned by MediaPipe Face Mesh are captured and stored as reference points. These four reference points define the boundary of the candidate's on-screen gaze region in landmark space. During live monitoring, the raw eye-landmark position for each frame is mapped into this calibrated region (interpolated against the four corner references) to produce a

normalized gaze score; a gaze score that falls outside the calibrated on-screen boundary indicates that the candidate is looking away from the screen and is used to raise a gaze-deviation event once it persists beyond the temporal-smoothing threshold.

For **head-pose calibration**, three reference images are captured: the candidate facing the *center* of the screen, the head turned toward the *left*, and the head turned toward the *right*. The yaw angle obtained from the PnP solution (Equation 2.1) at each of these three reference poses is recorded, with the center pose defining the neutral (zero) baseline and the left/right poses defining the acceptable yaw range for on-screen attention. Live head-pose readings are then expressed relative to this calibrated baseline and range rather than as raw, uncalibrated angles, which allows the head-pose threshold to adapt to each candidate’s natural neutral head position and webcam angle before a sustained deviation is flagged as an event.

This calibration step is performed once per session, immediately before the exam begins, and its reference values are reused for the gaze and head-pose checks throughout that session.

3.3.2. Object and Multiple-Person Detection Module

Unlike the gaze/pose module, the object detection module uses a **YOLOv8 model fine-tuned on a custom dataset** curated specifically for the exam-proctoring context (i.e., images/frames containing mobile phones, multiple persons, and typical exam-desk scenes). Fine-tuning on domain-specific data improves detection accuracy and reduces false positives relative to using the base COCO-pretrained weights alone.

3.3.3. Tab-Switching Detection Module

This module runs on the client side within the browser and listens for `visibilitychange` and `window blur/focus` events to detect when the candidate switches away from the exam tab or window. Each detected event is timestamped and sent to the backend as a suspicious-activity log entry.

3.3.4. Event Logging Engine

The event logging engine receives flagged events from all three detection modules (gaze/pose, object/person, tab-switch). Each event is timestamped, tagged with its source module, and stored along with a snapshot of the corresponding frame for administrator review.

3.4. Technology Stack

- **Frontend:** Next.js (client-side interface, webcam capture, tab-switch event listeners).
- **Backend:** FastAPI (AI processing pipeline, REST API endpoints).
- **Database:** PostgreSQL (user accounts, exam data, event logs).
- **Computer Vision Models:** MediaPipe Face Mesh (gaze/head-pose), YOLOv8 / Ultralytics (object and person detection).
- **Authentication:** JSON Web Tokens (JWT).
- **Model Training Environment:** Google Colab and Kaggle (free-tier GPU resources).

3.5. Development Methodology

As stated in Section 3.1, the Agile methodology is followed, with development organized into iterative cycles of planning, design, development, testing, and review for each module.

3.6. Data Handling

Input video frames are captured on the client and streamed to the backend, where they are processed frame-by-frame by the AI pipeline (Section 3.2.2) without persistent storage of raw video; only flagged event snapshots and metadata are stored in the PostgreSQL database. The custom dataset used to fine-tune the YOLOv8 model is described in detail in Section 4.4.1.

3.7. Testing and Validation

The system was tested under varying lighting conditions, webcam qualities, and scenarios, including: a single candidate with no malpractice, a candidate holding a mobile phone, a second person entering the frame, and a candidate switching browser tabs during the exam. Module-level (unit) testing was performed for each detection component individually, followed by integration testing of the full pipeline.

3.8. Deployment Setup

The system is designed to be CPU-friendly so that the AI processing pipeline can be deployed without dedicated GPU infrastructure. Model fine-tuning was carried out using

free-tier cloud GPU resources (Google Colab, Kaggle), while inference is intended to run on commodity server or client hardware, keeping deployment cost within a modest budget suitable for smaller institutions.

Chapter 4. Implementation Details

4.1. Requirements Specification

4.1.1. Functional Requirements

- **Registration and Login:** Students and administrators can register and authenticate into the system.
- **Exam Creation and Management:** Admins can create, configure, and manage exams.
- **Live Video Monitoring:** The system captures the student's webcam feed during the exam session.
- **Gaze and Head Pose Tracking:** The system continuously estimates gaze direction and head orientation using MediaPipe.
- **Object and Person Detection:** The system detects mobile phones and counts the number of persons visible using the fine-tuned YOLOv8 model.
- **Tab-Switch Detection:** The system detects and logs browser tab or window focus changes.
- **Event Logging and Alerts:** The system logs flagged events from each detection module and raises an alert whenever a module's detection threshold is exceeded.
- **Reporting:** Admins can view logs, flagged events, and generate exam reports.

4.1.2. Non-Functional Requirements

- **Performance:** The system processes video frames with minimal latency to support real-time monitoring.
- **Scalability:** The system supports multiple concurrent exam sessions.
- **Reliability:** The system degrades gracefully under poor network or lighting conditions.
- **Security:** Communication between client and server is encrypted; access is authenticated via JWT.
- **Usability:** The interface is designed to be intuitive for both students and administrators.

- **Maintainability:** The modular architecture allows independent updates to the detection models without affecting other components.

4.2. System Overview

The implemented system realizes the methodology of Chapter 3 as three independent, CPU-efficient detection modules (gaze/head pose, object/person, tab-switch) feeding into a shared event logging engine, all coordinated through the FastAPI backend and presented via the Next.js frontend, as summarized in the use case diagram (Figure 4.1) and activity diagram (Figure 4.2).

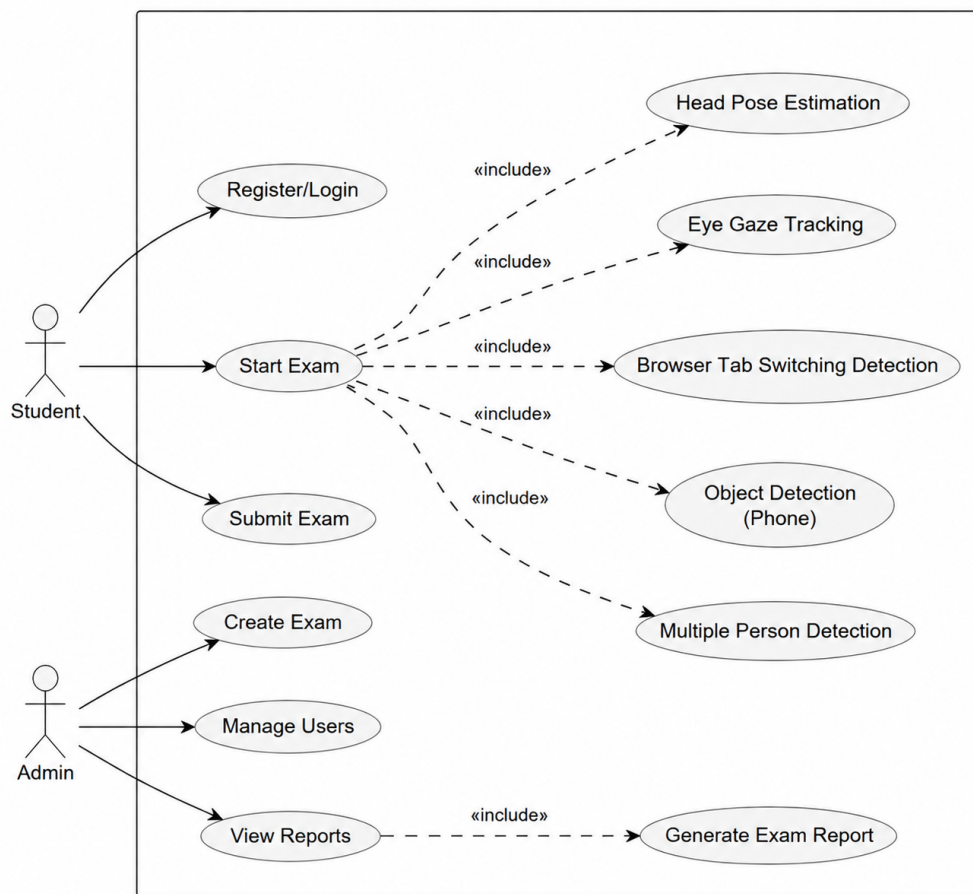


Figure 4.1: Use case diagram for VisionProctor

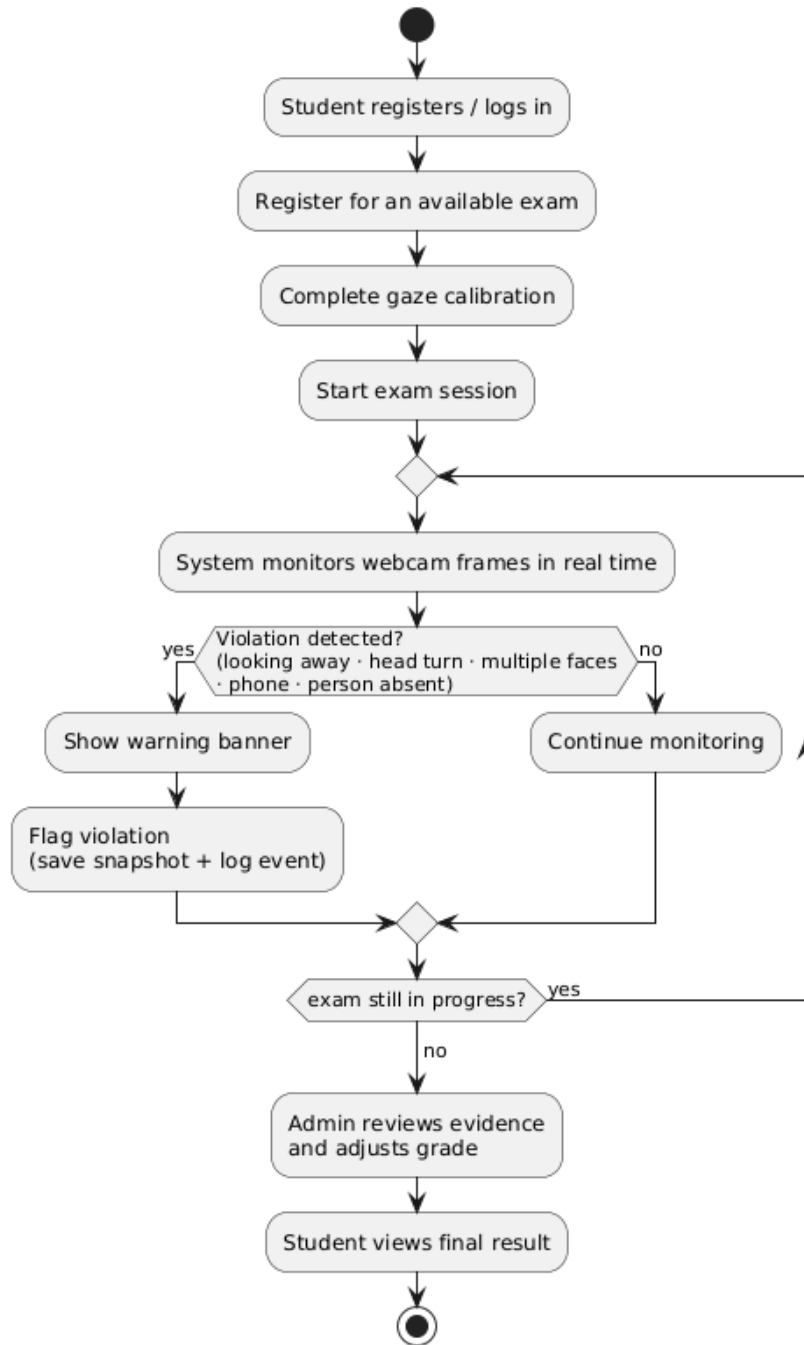


Figure 4.2: Activity diagram for VisionProctor

4.3. Development Environment

- **Programming Languages:** Python (AI pipeline, backend), JavaScript/TypeScript (frontend).
- **Frameworks and Libraries:** FastAPI, Next.js, OpenCV, MediaPipe, Ultralytics YOLOv8, PostgreSQL.
- **Model Training Hardware:** Google Colab, NVIDIA Tesla T4 GPU.

- **Development Hardware:** Personal laptops (CPU-based development and testing of the inference pipeline).

4.4. System Implementation

4.4.1. Dataset

A custom dataset was curated and annotated for fine-tuning the YOLOv8 object detection model, containing labelled instances of mobile phones and multiple persons in typical exam-desk webcam scenarios. The pretrained MediaPipe Face Mesh model was used as-is (without additional fine-tuning) for gaze and head-pose estimation. The dataset was obtained from Roboflow Universe [8].

Table 4.1: Dataset summary

Class	Number of Instances	Split (Train/Val/Test)
Person	13,334	Roboflow default split
Cell Phone	3,437	Roboflow default split
Book	1,935	Roboflow default split
TV	1,103	Roboflow default split
Laptop	702	Roboflow default split
Headphone	673	Roboflow default split

The total number of images in the dataset is 15,489 (640×640 , auto-oriented). Only the *person* and *cell phone* classes were used for this project’s multiple-person and mobile-detection modules.

4.4.2. Dataset Preprocessing

Images were resized and normalized to match the YOLOv8 input resolution. Standard data augmentation techniques, such as random horizontal flipping, scaling, mosaic augmentation, and brightness/contrast jittering, were applied to improve generalization under varying lighting conditions.

4.4.3. Model Selection

- **Gaze and Head Pose:** Pretrained MediaPipe Face Mesh (468 landmarks), selected for its real-time, CPU-efficient performance and robustness across head poses, requiring no additional training.
- **Object / Person Detection:** YOLOv8, fine-tuned on the custom dataset, selected

for its balance of speed and accuracy, making it suitable for real-time inference on modest hardware.

4.4.4. Training Configuration (YOLOv8 Fine-Tuning)

- **Base Model:** YOLOv8s (pretrained on COCO); the small variant was chosen over the nano variant for improved accuracy while remaining fast enough for near-real-time inference.
- **Framework:** Ultralytics.
- **Optimizer:** Auto (Ultralytics default optimizer selection between SGD and AdamW).
- **Learning Rate:** Default Ultralytics initial learning rate ($lr = 0.01$).
- **Epochs:** 100.
- **Batch Size:** 32.
- **Image Size:** 640×640 .
- **Dataloader Workers:** 4.
- **Early Stopping Patience:** 20 epochs without improvement.
- **Dataset Caching:** Enabled (`cache=True`) for faster training.
- **Augmentations:** Mosaic, horizontal flip, HSV jitter (Ultralytics default augmentation pipeline).
- **Hardware:** Google Colab, NVIDIA Tesla T4 GPU.

The classification component of the training objective follows the cross-entropy formulation given in Equation 2.2.

4.4.5. Real-Time Performance Considerations

Since the system is intended for live proctoring, inference latency and frame-rate throughput are critical. Techniques such as model quantization and adjustable frame sampling were explored to keep CPU usage low while maintaining acceptable detection responsiveness.

4.5. Interface and Visualization

4.5.1. Security Measures

The system uses JSON Web Tokens (JWT) for authentication. Upon successful login, the backend issues a JWT, which the client includes in the `Authorization` header of subsequent requests using the Bearer scheme. All communication between client and server occurs over HTTPS, and sensitive data such as passwords are hashed using a secure algorithm (bcrypt) before storage.

4.5.2. Actors and Interfaces

The primary actors of the system are the **Student** and the **Admin**. Students can register/login, undergo the exam-start procedure (which internally includes head-pose estimation, eye-gaze tracking, browser tab-switch detection, object detection, and multiple-person detection), and submit exams. Admins can create exams, manage users, and view generated reports (Figure 4.1).

4.6. Deployment and Execution

The client (Next.js) is served to the student’s browser, which establishes a webcam stream to the FastAPI backend over an encrypted connection. The backend, once started, loads the pretrained MediaPipe model and the fine-tuned YOLOv8 weights into memory and begins processing incoming frames for each active exam session, logging flagged events to the PostgreSQL database and exposing an administrator reporting interface.

4.7. Challenges and Solutions

- **Challenge:** Detection accuracy degraded under varying lighting conditions.
Solution: Data augmentation (brightness/contrast jittering) was applied during YOLOv8 fine-tuning to improve robustness.
- **Challenge:** Momentary head movements produced false-positive alerts.
Solution: Temporal smoothing (minimum duration thresholds and exponential moving averages) was applied to each module’s raw signal before an event was logged.
- **Challenge:** Running both the facial landmark model and the object detector in real time was computationally demanding on CPU-only hardware.
Solution: Adjustable frame sampling and a lightweight YOLOv8 variant (YOLOv8s) were used to balance accuracy and inference speed.

- **Challenge:** Raw, uncalibrated eye-landmark and head-pose readings differed noticeably between candidates due to differences in seating position, webcam angle, and face geometry, causing inconsistent gaze/head-pose thresholds.

Solution: A per-session calibration step (Section 3.4.1.1) was introduced, in which the four screen-corner points are used to calibrate gaze and the center/left/right head images are used to calibrate head pose, so that subsequent detection is performed relative to each candidate's own calibrated baseline.

Chapter 5. Results and Discussion

5.1. Results

5.1.1. Object Detection Performance

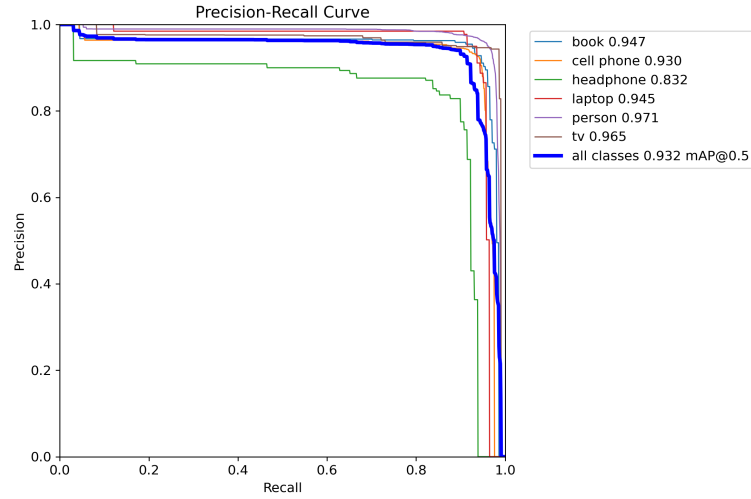


Figure 5.1: Precision-recall curve for the fine-tuned YOLOv8 model

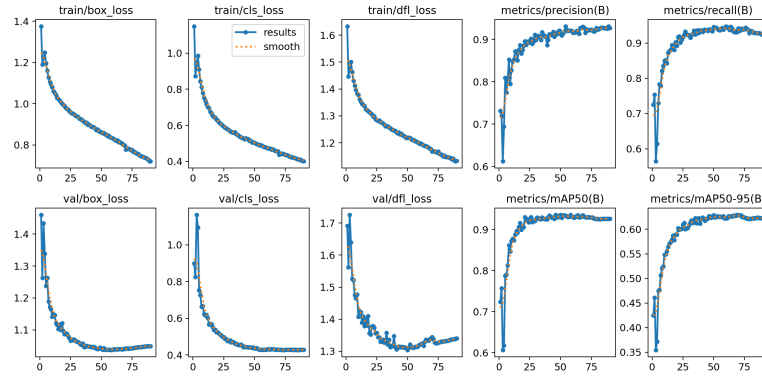


Figure 5.2: Training and validation loss curves (box, classification, and DFL losses) and object detection performance metrics (precision, recall, mAP@50, and mAP@50–95) during YOLOv8 fine-tuning

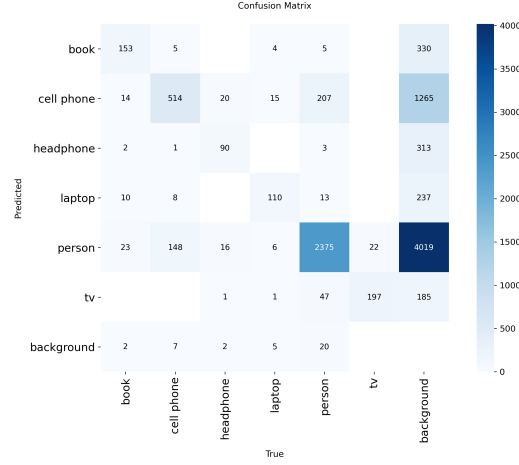


Figure 5.3: Confusion matrix for the fine-tuned YOLOv8 model on the test set

Table 5.1: YOLOv8 fine-tuning results

Metric	Value
Precision	0.92719
Recall	0.92608
mAP@0.5	0.9262
mAP@0.5:0.95	0.62223
F1 Score	0.92663

5.1.2. Calibration: Gaze and Head Pose Tracking

Before gaze and head-pose thresholds are applied for a session, the candidate completes the one-time calibration procedure described in Section 3.4.1.1. Figure 5.4 shows the eye-calibration step, in which the four corner points of the laptop screen are used as fixation targets and the corresponding eye landmarks are recorded as gaze reference points. Figure 5.5 shows the head-pose calibration step, in which center, left, and right head images are captured and used to establish the neutral baseline and acceptable yaw range for head-pose tracking. These calibrated reference values are then used throughout the session to compute normalized gaze and head-pose scores, in place of raw, uncalibrated landmark positions.

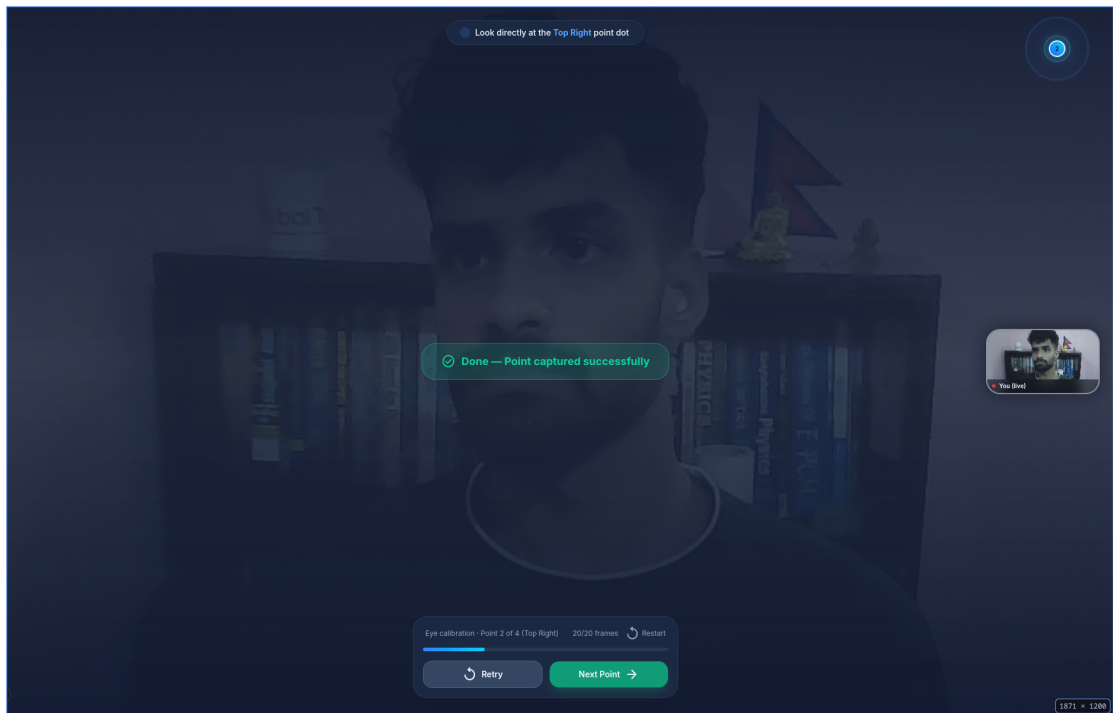


Figure 5.4: Eye calibration using the four laptop-screen corner points as gaze reference endpoints

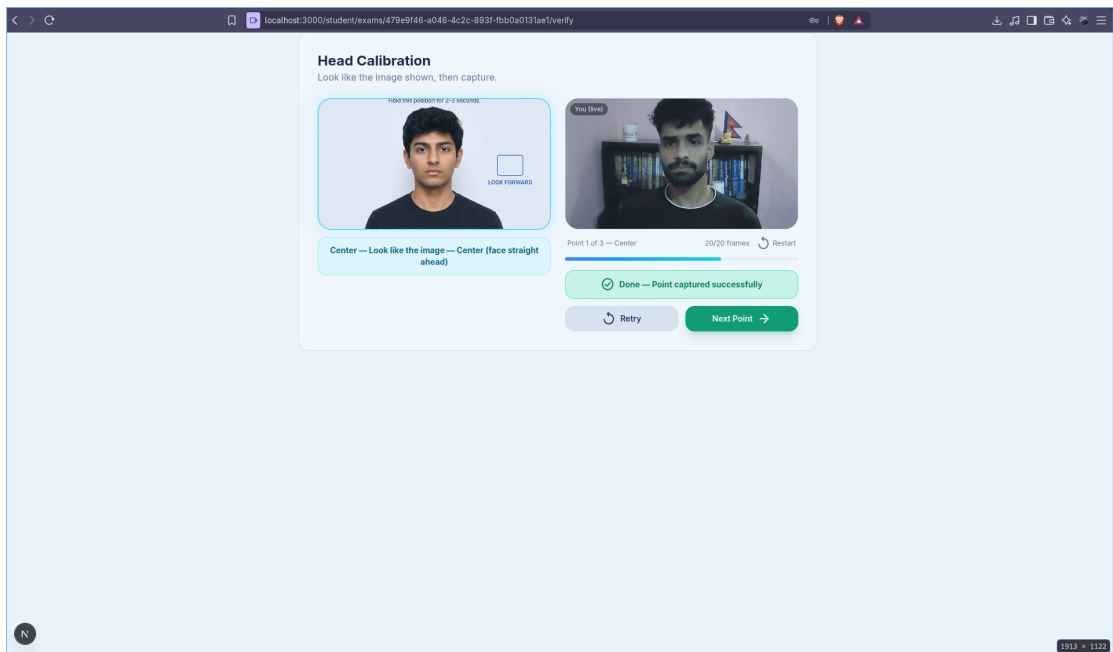


Figure 5.5: Head-pose calibration using center, left, and right reference head images

5.1.3. Qualitative Results: Gaze and Head Pose Tracking

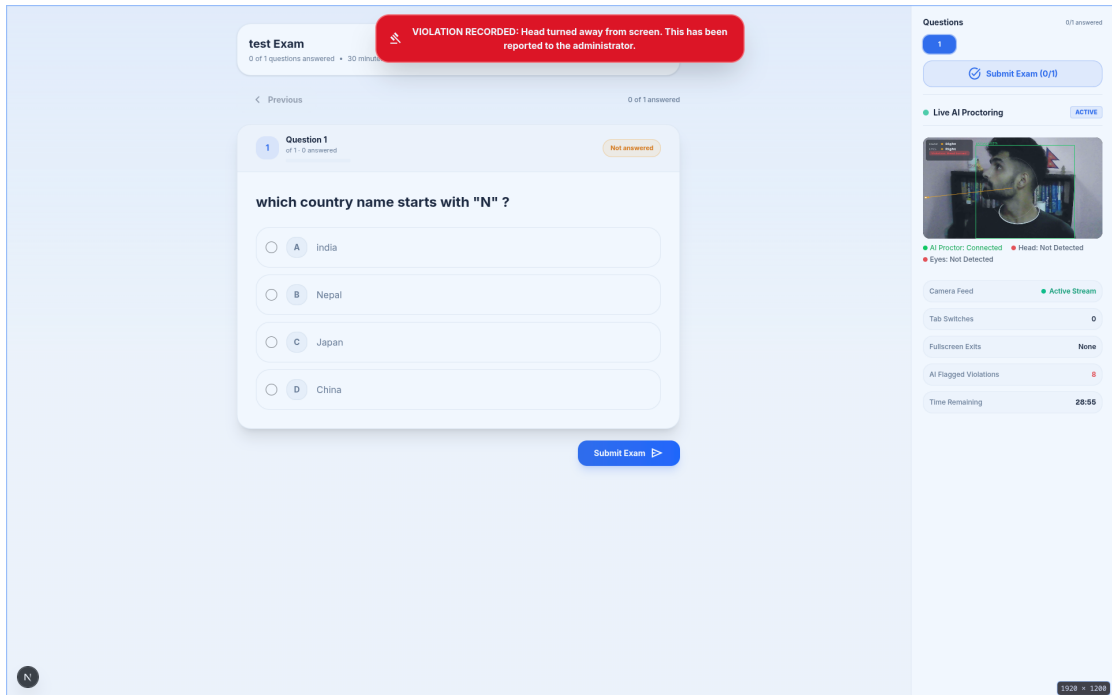


Figure 5.6: Sample frames with head-pose estimation

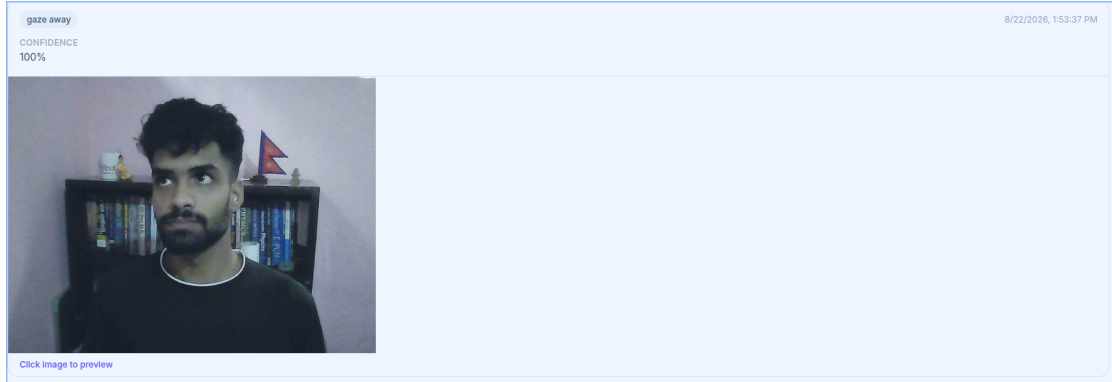


Figure 5.7: Sample frames with gaze deviation

5.1.4. Qualitative Results: Phone and Multiple-Person Detection

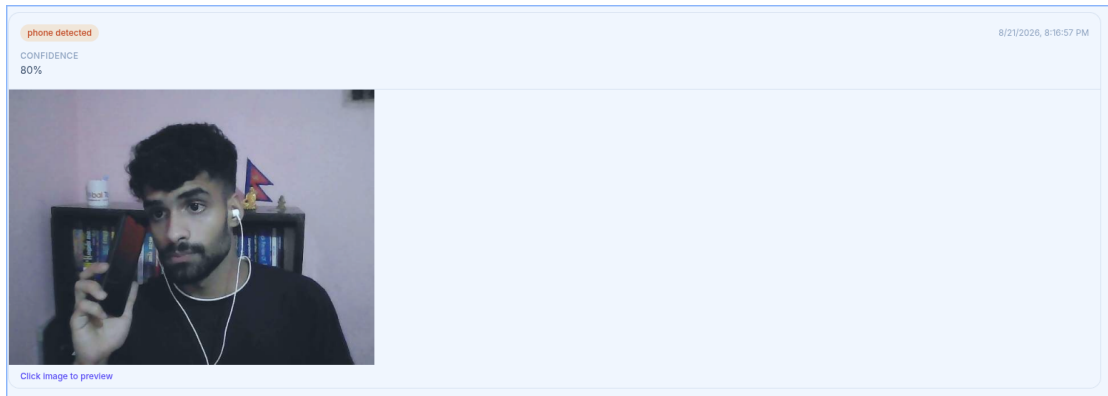


Figure 5.8: Sample detection result: mobile phone

5.1.5. Qualitative Results: Tab-Switch Detection

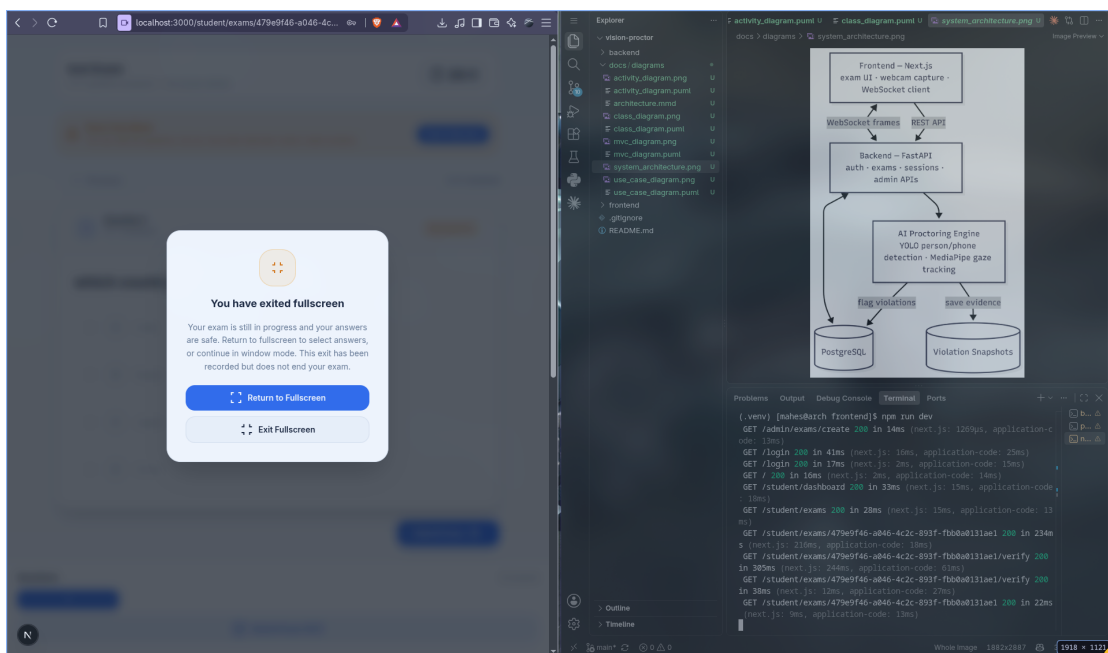


Figure 5.9: Live tab-switch detection

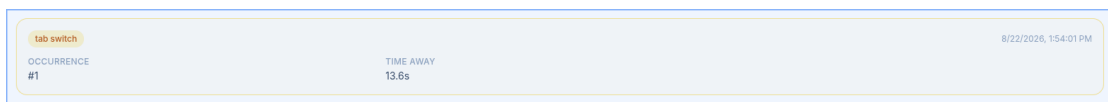


Figure 5.10: Tab-switch detection snapshot

5.1.6. Qualitative Results: Person Absent

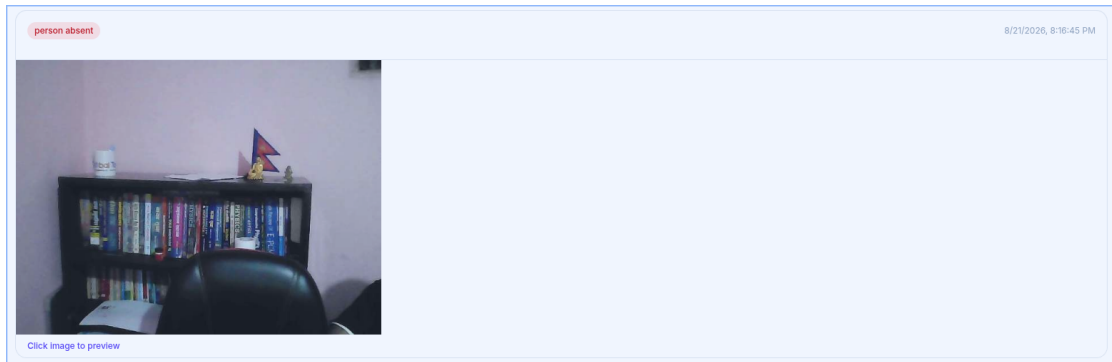


Figure 5.11: Person-absent snapshot

5.1.7. Event Log Summary

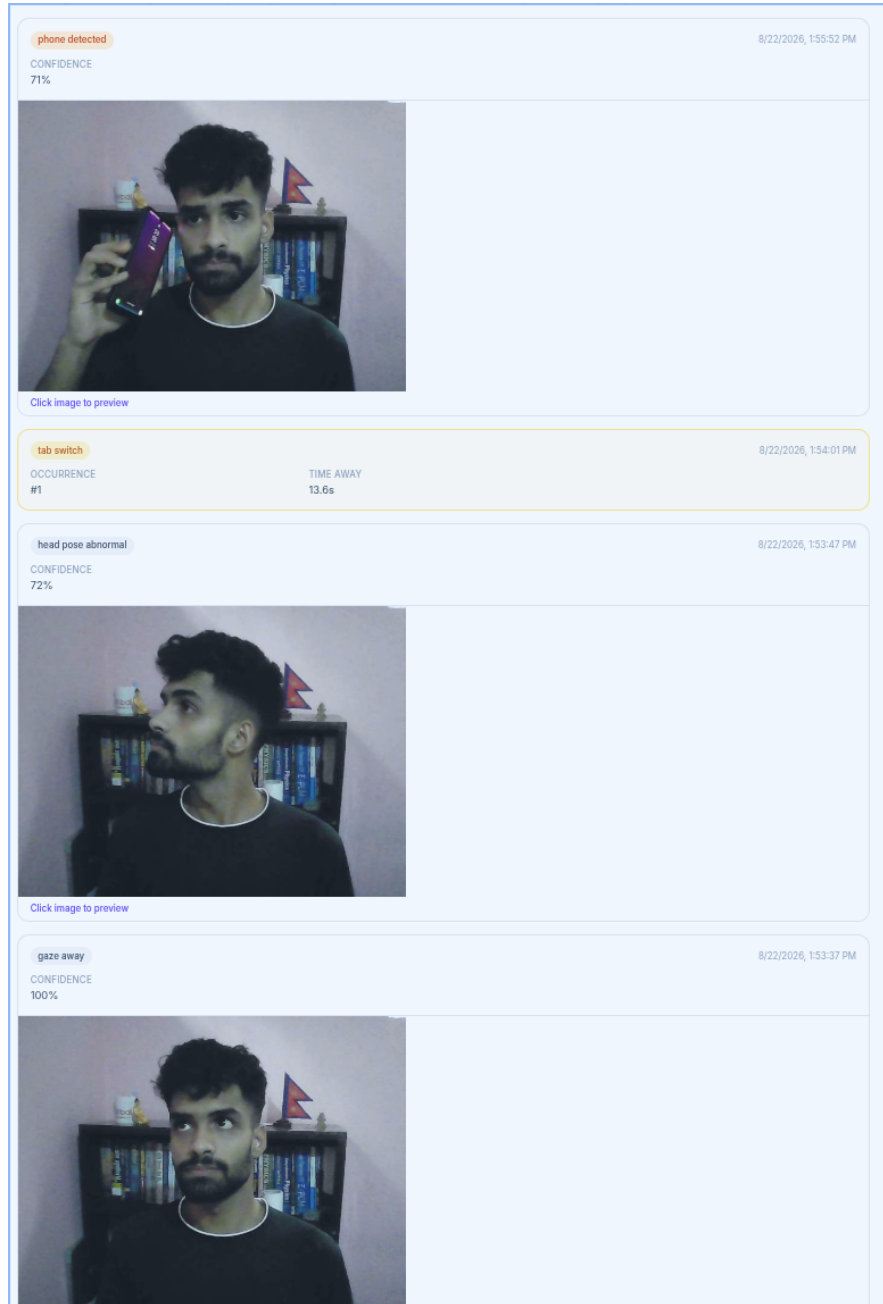


Figure 5.12: Logged suspicious events over time for a sample exam session

5.2. Discussion

The fine-tuned YOLOv8 model achieved a precision of 0.927, a recall of 0.926, and an mAP@0.5 of 0.926 (Table 5.1), indicating reliable detection of mobile phones and multiple persons under the tested conditions. The lower mAP@0.5:0.95 value (0.622) reflects the stricter localization requirement of this metric and suggests that bounding-box precision could be further improved with a larger or more diverse training set. The per-session calibration procedure (Section 3.4.1.1, Figures 5.4 and 5.5) was found to make the gaze

and head-pose thresholds substantially more consistent across candidates with different seating positions and webcam angles, compared to using fixed, uncalibrated thresholds. Qualitative results (Figures 5.6–5.12) show that the gaze/head-pose, object/person, and tab-switch modules each correctly flagged their respective malpractice scenarios during testing, and that the event logging engine successfully aggregated timestamped events from all three modules into a reviewable summary.

5.3. Comparative Analysis

Compared with the related approaches summarized in Table 2.1, VisionProctor integrates gaze/head-pose tracking, object/person detection, and browser-level tab-switch monitoring within a single modular system, whereas Proctorio and Respondus are closed-source and reported to have weaker object detection, and the CNN-based approach of Chen and Alvarez [7] focuses on a single behavioural signal without tab-switch monitoring. The measured detection performance of the fine-tuned YOLOv8 model (mAP@0.5 of 0.926) is comparable to, or exceeds, the approximately 91% accuracy reported for the CNN-based approach of Chen and Alvarez [7], while additionally covering object/person detection and browser-activity monitoring that are not addressed by that approach.

Chapter 6. Conclusions and Recommendations

6.1. Conclusions

This report has presented **VisionProctor**, an AI-based online proctoring system designed to address the growing need for scalable, automated exam-integrity monitoring. By combining a pretrained MediaPipe Face Mesh model for real-time gaze and head-pose tracking with a YOLOv8 object detection model fine-tuned specifically for detecting mobile phones and multiple persons, along with a browser-level tab-switching detection module, the system independently monitors and logs suspicious behaviour from each of these signals. This modular approach, combined with per-session calibration and per-module temporal smoothing, is intended to reduce the false-positive rates commonly observed in single-signal proctoring systems, while remaining lightweight and CPU-friendly compared with expensive commercial alternatives such as Proctorio and Respondus.

The proposed client-server architecture, built on Next.js and FastAPI with a PostgreSQL backend, provides a scalable foundation for real-time video streaming, exam management, and administrative reporting. The evaluation results (Chapter 5) indicate that the fine-tuned YOLOv8 model achieves high detection performance (mAP@0.5 of 0.926), and qualitative testing confirms that all three detection modules function as intended.

6.2. Recommendations / Future Enhancements

1. **Audio-Based Cheating Detection:** Incorporate audio analysis to detect conversations, whispering, or other suspicious sounds during the exam.
2. **Mouth Movement Detection:** Use facial landmarks to monitor lip movement and detect continuous speaking, which may indicate that the candidate is talking to someone off-screen.
3. **Model Optimization:** Further fine-tune and quantize the YOLOv8 model on an expanded, more diverse dataset to improve accuracy and reduce inference latency.
4. **Edge Deployment:** Optimize the pipeline for on-device (client-side) inference to reduce server load and improve data privacy.
5. **Expanded Dataset:** Collect and annotate a larger and more diverse dataset covering different lighting conditions, camera qualities, and demographics.

6. **Adaptive Thresholding:** Explore adaptive, per-candidate detection thresholds for each module to further reduce false positives.
7. **Unified Suspicion Scoring:** As a future extension, explore combining the independently logged events from each module into a single weighted suspicion score to simplify administrator review.

References

- [1] C. Lugaresi *et al.*, “MediaPipe: A Framework for Building Perception Pipelines,” *arXiv preprint arXiv:1906.08172*, 2019. [Online]. Available:<https://arxiv.org/abs/1906.08172>
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” *GitHub repository*, 2023. [Online]. Available:<https://github.com/ultralytics/ultralytics>
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. [Online]. Available:https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html
- [4] T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context,” in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755. [Online]. Available:<https://www.microsoft.com/en-us/research/publication/microsoft-coco-common-objects-in-context/>
- [5] Proctorio, *Official Product Documentation*. [Online]. Available:<https://proctorio.com/support/hc>
- [6] Respondus, *LockDown Browser and Monitor Documentation*. [Online]. Available:<https://support.respondus.com/hc/en-us/categories/4409595254811-LockDown-Browser-Respondus-Monitor>
- [7] X. Chen and R. Alvarez, “CNN-Based Online Exam Proctoring.”
- [8] “Online Proctoring System Dataset,” *Roboflow Universe*, 2morrow Software Limited. [Online]. Available:<https://universe.roboflow.com/project-2morrow-software-limited/online-proctoring-system-x27ou>
- [9] Y. Atoum, L. Chen, A. X. Liu, S. D. H. Hsu, and X. Liu, “Automated Online Exam Proctoring,” *IEEE Transactions on Multimedia*, vol. 19, no. 7, pp. 1609–1624, 2017. [Online]. Available:<https://doi.org/10.1109/TMM.2017.2656064>
- [10] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “OpenFace 2.0: Facial Behavior Analysis Toolkit,” in *Proc. IEEE International Conference on Automatic Face and Gesture Recognition*, 2018. [Online]. Available:<https://doi.org/10.1109/FG.2018.00019>

Appendix 1

1.1. Sample Annotated Dataset Images

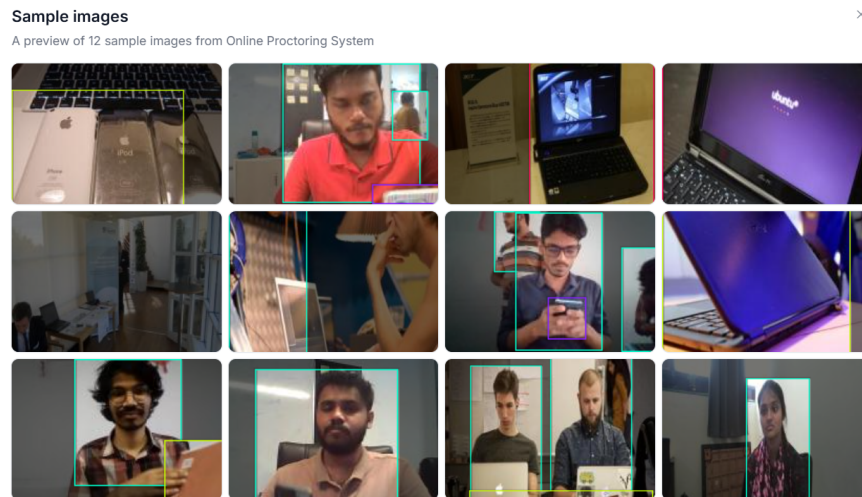


Figure 1.1: Sample annotated images from the custom object-detection dataset